In [2]:
```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

In [28]:
```
image = cv2.imread(r"C:\Users\USER\OneDrive - GJU\Desktop\4th Year\MI 2\Homeworks\Schweinfurt.jpg")
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
plt.imshow(image)
```

Out[28]: <matplotlib.image.AxesImage at 0x19483235e50>



In [105…
```
# Harris Corner Detection  x1 sacle
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
gray_float = np.float32(gray)
dst = cv2.cornerHarris(gray_float, blockSize=3, ksize= 3, k=0.04)
image_with_corners = image.copy()
image_with_corners[dst > 0.01 * dst.max()] = [255, 0, 0]
```

```
print("dst max:", dst.max())
print("dst mean:", dst.mean())
plt.figure(figsize=(10, 8))
plt.imshow(image_with_corners)
plt.title('Harris Corner Detection')
plt.axis('off')
plt.show()
```

```
dst max: 1.201412e+08
dst mean: 50894.625
```



In [106…

```
# Harris Corner Detection x1.5 scale
resized_image = cv2.resize(image, None, fx=1.5, fy=1.5, interpolation=cv2.INTER_LINEAR)
gray_resized = cv2.cvtColor(resized_image, cv2.COLOR_BGR2GRAY)
gray_resized_float = np.float32(gray_resized)
dst_resized = cv2.cornerHarris(gray_resized_float, blockSize=3, ksize= 3, k=0.04)
resized_image_with_corners = resized_image.copy()
resized_image_with_corners[dst_resized > 0.01 * dst_resized.max()] = [255, 0, 0]
```

```
plt.figure(figsize=(10, 8))
plt.imshow(resized_image_with_corners)
plt.title('Harris Corner Detection x1.5 Scale')
plt.axis('off')
```

Out[106… (np.float64(-0.5), np.float64(2429.5), np.float64(1619.5), np.float64(-0.5))



Even when the output image is displayed without any scaling in VS Code, the ×1.5 scaled image still results in fewer detected corners. This behavior is consistent with the fact that the Harris corner detector is not scale-invariant or covariant.

In [116…

```
# DoG blob detection
blur1 = cv2.GaussianBlur(gray, (0, 0), sigmaX=5.0)
blur2 = cv2.GaussianBlur(gray, (0, 0), sigmaX=9.0)
dog = blur1.astype(np.float32) - blur2.astype(np.float32)
dog_vis = cv2.normalize(dog, None, 0, 255, cv2.NORM_MINMAX, cv2.CV_8U)

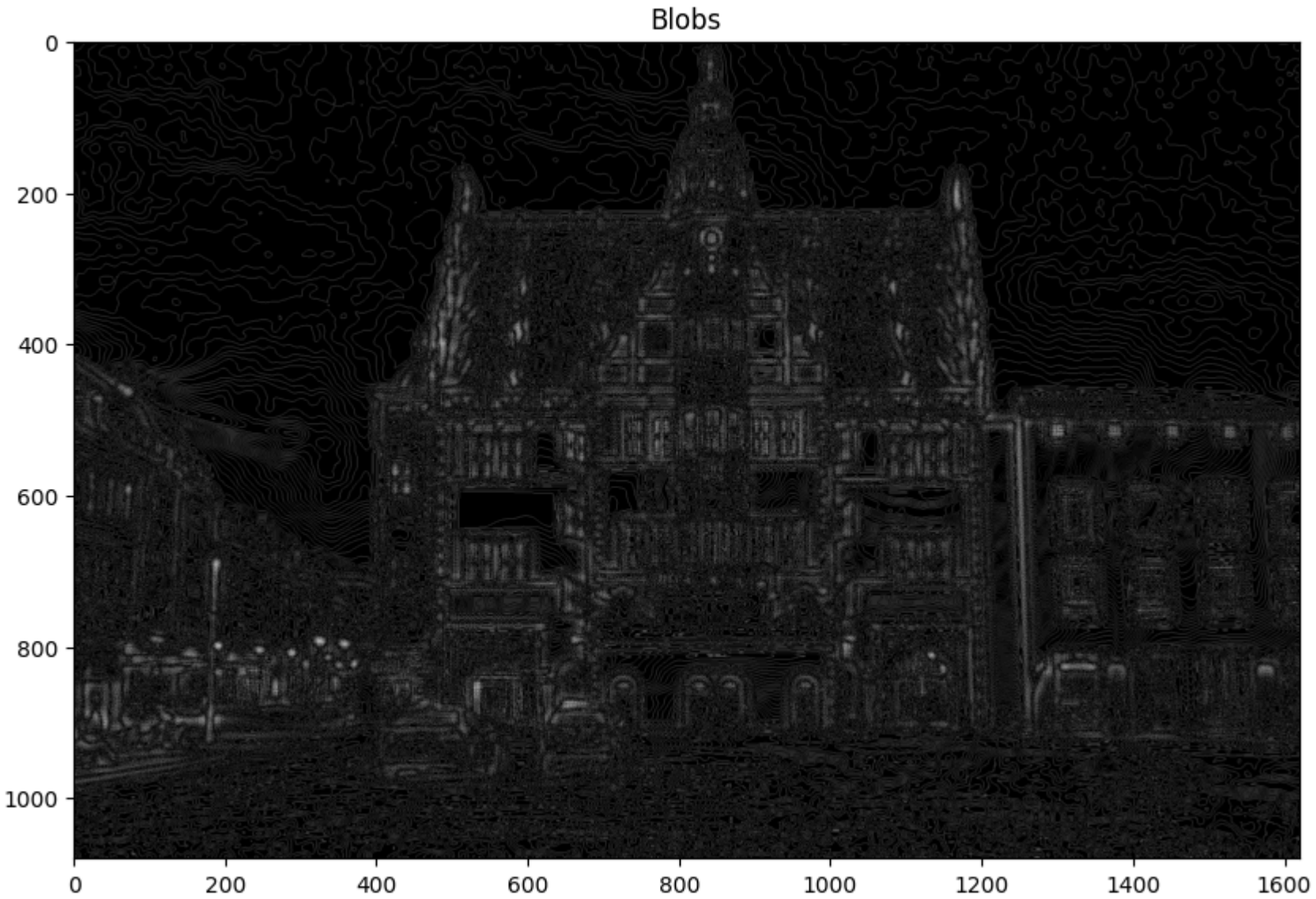
plt.figure(figsize=(10, 8))
plt.imshow(dog_vis, cmap='gray')
plt.title('Difference of Gaussians (DoG) Map')
plt.show()
```



In [ ]:
```
#blob detection using Laplacian of Gaussian (LoG) on x1 scale
lap_blob = cv2.Laplacian(blob_blur, cv2.CV_64F, ksize= 3)
lap_blob = cv2.convertScaleAbs(lap_blob)
plt.figure(figsize=(10, 8))
```

```
plt.imshow(lap_blob, cmap='gray')
plt.title('Laplacian: Blobs 1 scale')
```

Out[ ]: Text(0.5, 1.0, 'Blobs')



In [ ]:
```
# blob detection with Laplacian at 1.5 scale
scale = 1.5
h, w = gray.shape
gray_up = cv2.resize(gray, (int(w * scale), int(h * scale)),interpolation=cv2.INTER_CUBIC)


blob_blur_up = cv2.GaussianBlur(gray_up, (0, 0), 4.8)
lap_up = cv2.convertScaleAbs(cv2.Laplacian(blob_blur_up, cv2.CV_64F, ksize=3))

plt.figure(figsize=(10, 8))
plt.imshow(lap_up, cmap='gray')
plt.title('Laplacian: Upscaled x1.5')
plt.axis('off')
plt.tight_layout()
plt.show()
```



not much changed, but the scaled version seemed to ignore weak blobs and highlighted the stronger.